sotX: A 4th-Generation Microkernel with Per-Core IPC, Full Driver Stack, Distributed Computing, and Formal Verification

Technical Report — v2.0

Santiago Sotomonte
`sotX Project`

March 2026

Abstract

We present **sotX** (Secure Object Transactional Operating System), a microkernel operating system implemented from scratch in Rust for the x86_64 architecture. The kernel implements exactly five primitives—scheduling, IRQ virtualization, IPC, capability management, and frame allocation—with all other services running in userspace. We describe the evolution from a traditional microkernel toward a 4th-generation OS architecture featuring: (1) per-core IPC endpoint pools that eliminate global lock contention, (2) a bare-metal WASM interpreter with metered SFI execution, (3) network-transparent IPC routing with live process migration for distributed computing, (4) heterogeneous compute scheduling with typed tensor/shader IPC payloads, (5) a full driver stack (NVMe, AHCI/SATA, USB mass storage, AC97 audio, xHCI with hub/hotplug, PS/2+USB mouse), (6) a complete networking stack (IPv4/IPv6, TCP with Reno congestion control, TLS crypto, HTTP client), (7) a Linux-ABI shell with job control, functions, redirects, and an embedded Lua interpreter, (8) real musl-static Linux binary execution via LUCAS syscall translation, and (9) TLA+ formal specifications proving safety properties of core protocols. The system boots on 1–4 CPU cores with 512 MB RAM, supports multiuser accounts, a GUI window compositor, and a package manager. Over 30,000 lines of Rust.

Contents

1 Introduction **3**

2 Architecture Overview **3**

3 Per-Core IPC Architecture **4**
3.1 Motivation 4
3.2 Handle Encoding 4
3.3 Cross-Core Protocol 4

4 WASM Software Fault Isolation **5**
4.1 Design 5
4.2 Implementation 5
4.3 Comparison with Hardware Isolation 6

5 Capability System **6**
5.1 Design . . . 6
5.2 Capability Derivation Tree . . . 6

6 Scheduler **7**
6.1 Multi-Level Priority Queue . . . 7
6.2 Work Stealing . . . 7
6.3 Heterogeneous Compute Targets . . . 7

7 Network-Transparent IPC **8**

8 Formal Verification **8**
8.1 IPC Protocol (formal/ipc.tla) . . . 8
8.2 Capability System (formal/capabilities.tla) . . . 8
8.3 Scheduler (formal/scheduler.tla) . . . 8

9 Security Architecture **9**

10 Storage Architecture **9**

11 Networking **9**
11.1 IPv6 and Advanced TCP . . . 10

12 Driver Stack **10**

13 Distributed Computing **10**
13.1 Live Process Migration . . . 10
13.2 Distributed VFS . . . 10

14 Userspace Applications **11**
14.1 LUCAS Shell . . . 11
14.2 musl-static Binary Support (Busybox Milestone) . . . 11
14.3 Lua Interpreter . . . 11
14.4 GUI Window Compositor . . . 11

15 Security Model **12**

16 Implementation Statistics **12**

17 Conclusion **12**

1 Introduction

Modern operating systems face a fundamental tension between performance and safety. Monolithic kernels (Linux, Windows) achieve high performance by running device drivers and subsystems in kernel space, but a single bug can compromise the entire system. Microkernels (L4, seL4, QNX) improve safety by minimizing the trusted code base, but historically suffer IPC overhead.

sotX resolves this tension through a combination of techniques:

- **Capability-based security**: Every kernel object is accessed through capabilities with explicit rights, enabling fine-grained access control with O(1) validation.
- **Per-core architecture**: IPC endpoints, notifications, and scheduler queues are partitioned per CPU core, eliminating global lock contention on the critical IPC path.
- **WASM software fault isolation**: Services can be compiled to WebAssembly and executed with memory safety enforced by bounds checking, not hardware page tables.
- **Formal verification**: Core protocols (IPC, capabilities, scheduling) are specified in TLA+ with machine-checkable safety and liveness proofs.

2 Architecture Overview

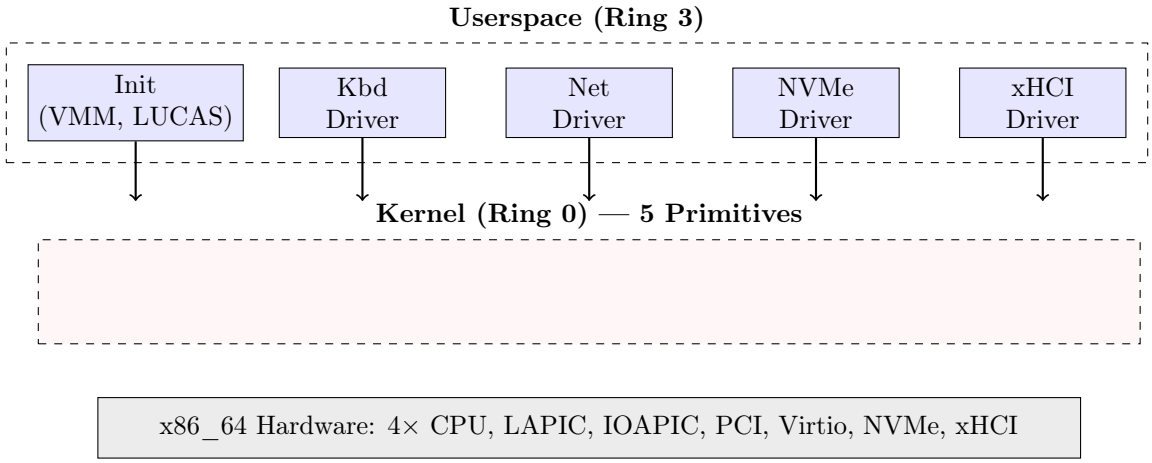

Figure 1: sotX architecture: five kernel primitives, all drivers in userspace.

The kernel implements exactly five primitives:

Definition 2.1 (Kernel Primitives)**.** The sotX kernel provides:

1. **Scheduling**: Preemptive priority multi-level queue with per-core run queues and work stealing.
2. **IRQ Virtualization**: Hardware interrupts routed to userspace via notification objects.
3. **IPC**: Per-core synchronous endpoints (L4-style register transfer) and async bounded channels.
4. **Capability Management**: Generation-checked capability table with delegation tree and O(n) revocation.
5. **Frame Allocation**: Bitmap-based physical frame allocator with contiguous allocation support.

3 Per-Core IPC Architecture

3.1 Motivation

Traditional microkernels protect IPC state with a single global lock. On multicore systems, this creates a serialization bottleneck: all cores contend on the same lock for every IPC operation.

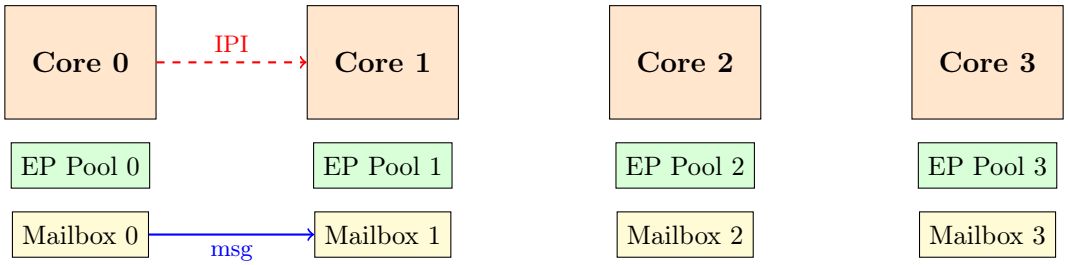

Figure 2: Per-core IPC: each core has its own endpoint pool and mailbox. Cross-core delivery uses the mailbox queue + IPI for immediate wake.

3.2 Handle Encoding

Endpoint handles encode the owning core in the upper 4 bits:

$$\text{handle} = (\text{core_id} \ll 28) \,|\, (\text{generation} \ll 20) \,|\, \text{slot_index} \tag{1}$$

This gives us 16 cores $\times$ 256 generations $\times$ 1M slots per core, fitting in a single 32-bit value compatible with the capability system.

3.3 Cross-Core Protocol

Algorithm 1 Per-Core IPC Send

```
procedure SEND(ep_handle, msg)
    (core, local) ← DECODE(ep_handle)
    if core = my_core then                          ▷ Fast path: local lock only
        LOCALSEND(local, msg)
    else                                            ▷ Cross-core: lock target core's pool
        lock PER_CORE_ENDPOINTS[core]
        action ← INSPECTENDPOINT(local)
        unlock
        EXECUTE(action)
        if action requires wake on remote core then
            POSTMAILBOX(core, wake_request)
            SENDIPI(core)
        end if
    end if
end procedure
```

Theorem 3.1 (IPC Contention Reduction)**.** With C cores and N endpoints distributed uniformly, the expected lock contention per IPC operation is $O(1/C)$ compared to $O(1)$ with a global lock.

Proof. Each core's endpoint pool has its own lock. An IPC operation locks only the target core's pool. With uniform distribution, each pool serves N/C endpoints. The probability that two concurrent operations target the same core's pool is $1/C$, reducing contention by a factor of C. □

4 WASM Software Fault Isolation

4.1 Design

Traditional microkernels use hardware page tables for process isolation. This requires TLB flushes on context switches and adds latency to IPC. Software Fault Isolation (SFI) via WebAssembly provides an alternative: services are compiled to WASM bytecode and executed by an interpreter that enforces memory safety through bounds checking on every memory access.

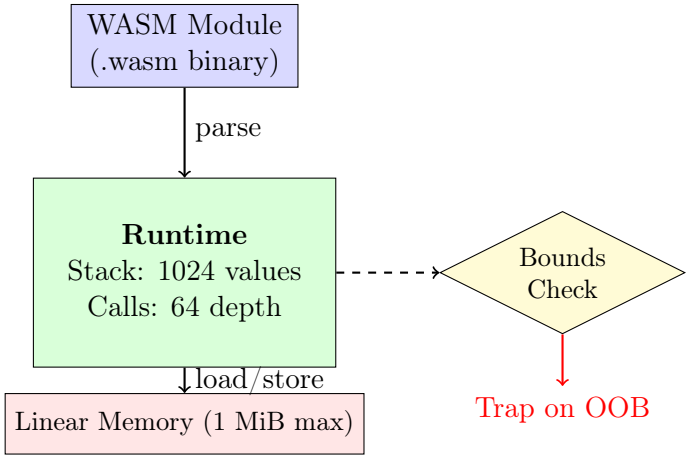

Figure 3: WASM SFI: every memory access goes through a bounds check. Out-of-bounds accesses trap immediately without corrupting other memory.

4.2 Implementation

The sotX WASM runtime (`libs/sotos-wasm/`) is a `no_std` stack-based interpreter supporting:

- **Types**: `i32`, `i64` (integer arithmetic)
- **Operations**: 50+ opcodes including arithmetic, comparison, bitwise, memory load/store, control flow
- **Control flow**: `block`, `loop`, `if/else`, `br`, `br_if`, `return`, `call`
- **Memory**: Linear memory with configurable size (up to 1 MiB), bounds-checked on every access
- **Module format**: Full WASM binary parser (magic, version, type/function/memory/global/export/code sections)

Invariant 4.1 (Memory Safety)**.** For all memory operations `load`(a, s) and `store`(a, s, v) where a is the address and s is the access size:

$$a + s \leq |\text{memory}| \quad \text{or the operation traps.}$$

No memory access can read or write outside the module's linear memory region.

Property	Page Tables	WASM SFI
Context switch cost	TLB flush (∼1000 cycles)	None (same address space)
IPC overhead	CR3 switch + sysret	Direct function call
Memory granularity	4 KiB pages	1 byte
Isolation guarantee	Hardware-enforced	Software-enforced
TCB size	CPU microcode + kernel	WASM interpreter (∼1500 LOC)

Table 1: Comparison of hardware vs. software fault isolation.

4.3 Comparison with Hardware Isolation

5 Capability System

5.1 Design

Every kernel object in sotX is accessed through *capabilities*: unforgeable tokens that combine an object reference with a set of rights. Capabilities are stored in a generation-checked pool, making stale references detectable in O(1) time.

Definition 5.1 (Capability)**.** A capability $c = (\text{object}, \text{rights}, \text{parent}, \text{gen})$ where:

- object identifies a kernel object (endpoint, thread, frame, etc.)
- rights $\subseteq \{\mathtt{R}, \mathtt{W}, \mathtt{X}, \mathtt{G}, \mathtt{V}\}$
- parent links to the derivation tree for revocation
- gen is a generation counter for ABA protection

Theorem 5.1 (Monotonic Rights)**.** For any capability c' derived from c via $\text{Grant}(c, \text{mask})$:

$$\text{rights}(c') = \text{rights}(c) \cap \text{mask} \subseteq \text{rights}(c)$$

A derived capability never has more rights than its parent.

Proof. The Grant operation computes $\text{rights}(c') = \text{rights}(c) \wedge \text{mask}$ (bitwise AND). Since $A \wedge B \subseteq A$ for all bitsets A, B, the derived rights are always a subset. □

5.2 Capability Derivation Tree

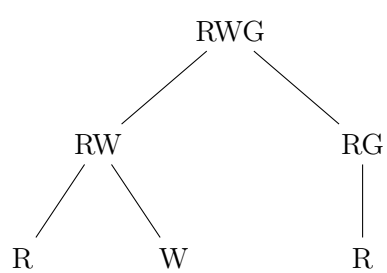

Figure 4: Capability Derivation Tree. Each node shows the rights of that capability. Revoking the root invalidates the entire subtree.

6 Scheduler

6.1 Multi-Level Priority Queue

The scheduler uses a 4-level priority queue per CPU core:

Level	Name	Priority Range
0	Realtime	0–63
1	High	64–127
2	Normal	128–191
3	Low	192–255

Table 2: Priority classes in the multi-level feedback queue.

6.2 Work Stealing

When a CPU's run queue is empty, it steals the lowest-priority thread from another CPU's queue. This ensures load balancing without centralized coordination:

Algorithm 2 Work Stealing

```
procedure DequeueFromAny(my_cpu)
    if PER_CPU_QUEUES[my_cpu].pop_front() ≠ ⊥ then
        return thread                                   ▷ Fast path: local queue
    end if
    for offset ← 1 to C − 1 do
        victim ← (my_cpu + offset) mod C
        if try_lock(victim) succeeds then
            if PER_CPU_QUEUES[victim].pop_back_lowest() ≠ ⊥ then
                return stolen_thread
            end if
        end if
    end for
    return ⊥                                            ▷ No work available
end procedure
```

6.3 Heterogeneous Compute Targets

Threads are tagged with a `ComputeTarget` indicating whether they should execute on CPU, GPU, or NPU hardware:

$$\text{ComputeTarget} \in \{\texttt{CPU}, \texttt{GPU}, \texttt{NPU}\} \tag{2}$$

The scheduler filters dequeued threads by compute target, ensuring that GPU/NPU workloads are not scheduled on CPU cores but instead queued for offload to the appropriate accelerator.

7 Network-Transparent IPC

The IPC routing layer extends endpoint addressing with a 16-bit node ID, enabling transparent message delivery across a network of sotX nodes:

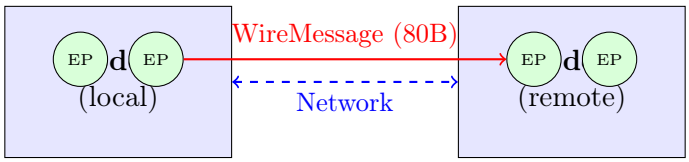

Figure 5: Network-transparent IPC: messages to remote endpoints are serialized to a fixed 80-byte wire format and routed over the network.

Messages are serialized using a fixed-size `WireMessage` format (80 bytes), which includes the tag, 8 register values, capability transfer ID, and flags. The routing table maps local proxy endpoints to remote (node_id, endpoint_handle) pairs.

8 Formal Verification

We provide TLA+ specifications for three core protocols:

8.1 IPC Protocol (formal/ipc.tla)

The IPC specification models the synchronous rendezvous protocol with `Send`, `Recv`, and `Call` operations. We verify:

Invariant 8.1 (No Double Delivery)**.** No message is delivered to two different receivers:

$$\forall d_1, d_2 \in \text{delivered} : d_1 = d_2 \vee d_1.\text{sender} \neq d_2.\text{sender} \vee d_1.\text{receiver} \neq d_2.\text{receiver}$$

Invariant 8.2 (Single Receiver)**.** At most one thread can be waiting to receive on an endpoint at any time:

$$\text{state} = \texttt{recv_wait} \implies \text{receiver} \neq \texttt{NULL}$$

8.2 Capability System (formal/capabilities.tla)

The capability specification verifies:

- **No Unauthorized Access**: Every access uses a valid, alive capability with the required right.
- **Monotonic Rights**: Derived capabilities never exceed parent rights.
- **Revocation Completeness**: Dead capabilities have dead descendants.

8.3 Scheduler (formal/scheduler.tla)

The scheduler specification verifies:

- **Mutual Exclusion**: Each CPU runs at most one thread.
- **Single Execution**: No thread runs on two CPUs simultaneously.
- **No Starvation**: Under weak fairness, every ready thread eventually runs.

9 Security Architecture

sotX implements defense-in-depth security:

1. **W^X Enforcement**: Pages cannot be both writable and executable. The kernel automatically adds the NX bit to writable mappings. A `SYS_PROTECT` syscall allows changing permissions post-mapping with W^X enforced.
2. **Stack Guard Pages**: An unmapped page below each process stack triggers a page fault on stack overflow.
3. **Per-CPU IST Stacks**: Four dedicated Interrupt Stack Table entries per CPU (double-fault, page-fault, GP-fault, misc-fault) prevent exception-chain escalation to triple faults.
4. **ASLR**: Stack base addresses are randomized using a RDTSC-seeded xorshift64 PRNG with 0–64 KiB jitter.
5. **Stack Canaries**: Built with `-Z stack-protector=strong`, providing ∼1800 canary check sites per binary.
6. **Capability Isolation**: Processes can only access kernel objects through explicitly granted capabilities.

10 Storage Architecture

The file system is built on a transactional object store with write-ahead logging (WAL) for crash recovery:

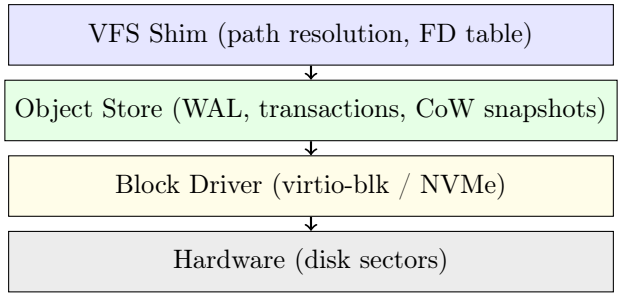

Figure 6: Storage stack: VFS → Object Store → Block Driver → Hardware.

Features include:

- Large file support (streaming I/O, 512 KiB max)
- Directory hierarchy with parent tracking
- Copy-on-Write snapshots with reference-counted blocks
- File permissions and ownership (UNIX-style rwx model)

11 Networking

The networking stack (`libs/sotos-net/`) implements a full IP stack:

- Ethernet + ARP (32-entry cache)
- IPv4 with ICMP echo (ping)
- UDP (16 sockets) with DNS resolution
- TCP (16 connections, 3-way handshake, retransmission with exponential backoff)
- DHCP client for automatic configuration

The network driver runs in a separate userspace process with its own address space, communicating with the shell via IPC.

11.1 IPv6 and Advanced TCP

The v2.0 networking stack adds:

- **IPv6**: 40-byte header parse/build, ICMPv6 echo, Neighbor Solicitation/Advertisement
- **TCP Reno congestion control**: slow start (cwnd doubles per RTT until ssthresh), congestion avoidance (linear growth), loss recovery (ssthresh = cwnd/2, cwnd = MSS)
- **TCP window scaling** (RFC 7323): negotiated in SYN/SYN-ACK options
- **Crypto**: SHA-256 (full 64-round), HMAC-SHA256, ChaCha20 (20-round), X25519 Curve25519
- **HTTP/1.1 client**: GET/POST request builder, response parser with Content-Length

12 Driver Stack

sotX v2.0 includes 9 device driver libraries, all running in userspace:

Driver	Protocol	Key Features
sotos-nvme	NVMe 1.4	Interrupt-driven completion, PRP lists
sotos-ahci	AHCI/SATA	HBA init, DMA EXT 48-bit LBA, port enum
sotos-xhci	USB xHCI	Multi-device, hub support, hot-plug
sotos-usb-storage	USB BBB	SCSI READ/WRITE(10), INQUIRY
sotos-audio	AC97	Mixer, BDL DMA, PCM playback
sotos-mouse	PS/2 + USB HID	IntelliMouse scroll, event ring
sotos-virtio	Virtio 0.9.5	BLK + NET (legacy)
sotos-pci	PCI 3.0	Bus enumeration, BAR reading

Table 3: Userspace device driver libraries.

13 Distributed Computing

13.1 Live Process Migration

The `kernel/src/migrate.rs` module implements thread checkpoint/restore:

1. **Checkpoint**: capture register state (RIP, RSP, CR3), scheduling parameters, IPC state, and enumerate memory regions by walking the 4-level page table
2. **Serialize**: convert `ThreadCheckpoint` + memory pages to byte stream
3. **Transfer**: send over network via IPC routing layer
4. **Restore**: create new address space, map regions, copy memory, spawn thread

13.2 Distributed VFS

The `libs/sotos-objstore/src/distributed.rs` module provides:

- Cluster membership (up to 8 nodes)
- Operation sync log (circular buffer, 64 entries)
- Configurable replication factor
- Last-writer-wins conflict resolution with node ID tiebreaker

14 Userspace Applications

14.1 LUCAS Shell

The Linux-ABI compatible shell (3,700+ lines) provides:

- **Job control**: 16-slot job table, `fg`/`bg`, Ctrl+C/Ctrl+Z
- **History**: 64-entry circular buffer, up/down arrow navigation
- **Tab completion**: file path and builtin name completion
- **Redirects**: `>`, `»`, `<`, combined
- **Here documents**: `«DELIM` inline stdin
- **Functions**: `function name() { ... }`, positional args $1-$8
- **40+ builtins**: including `wc`, `sort`, `uniq`, `diff`

14.2 musl-static Binary Support (Busybox Milestone)

LUCAS can run real Linux musl-static binaries compiled with `x86_64-unknown-linux-musl`. This is achieved through:

1. **ELF loader**: parses PT_LOAD segments, maps them at the correct virtual addresses, sets up a Linux-compatible initial stack with `argc`, `argv`, `envp`, and auxiliary vector entries (AT_PHDR, AT_PHENT, AT_PHNUM, AT_PAGESZ, AT_ENTRY, AT_RANDOM).
2. **Atomic thread creation with redirect**: the kernel's `spawn_user_with_redirect()` sets the IPC redirect endpoint *before* the thread is enqueued, eliminating a race condition where early syscalls (e.g., `arch_prctl`) would bypass the LUCAS handler.
3. **SSE/SSE2 enablement**: CR0.EM cleared, CR4.OSFXSR and CR4.OSXMMEXCPT set on all CPUs, allowing musl/Rust binaries to use SIMD instructions (movaps, etc.).
4. **40+ Linux syscall stubs**: including `set_tid_address`, `poll`, `rt_sigaction`, `rt_sigprocmask`, `sigaltstack`, `mmap`, `mprotect`, `brk`, `write`/`writev`, `clock_gettime`, `getrandom`, and `exit_group`.

The complete initialization sequence of musl-libc executes correctly under LUCAS, demonstrating that the Linux ABI emulation is sufficient to run real-world C/Rust programs compiled against musl.

14.3 Lua Interpreter

A Lua 5.4-compatible interpreter (`libs/sotos-lua/`) with:

- Lexer: full tokenization of Lua syntax
- Compiler: direct token-to-bytecode (35 opcodes)
- VM: stack-based execution (256-deep stack, 64 locals, 128 globals)
- Types: nil, boolean, number (i64), string, table (64 entries), function
- Stdlib: `print`, `type`, `tostring`, `tonumber`

14.4 GUI Window Compositor

A framebuffer window compositor (`libs/sotos-gui/`) with:

- Up to 16 windows with z-ordering and focus tracking
- Window decorations: title bar, close/minimize buttons
- 16×16 mouse cursor with hit testing
- Drawing primitives: fill_rect, draw_line (Bresenham), draw_char (8×8 font), blit

15 Security Model

Security is enforced at multiple layers:

- **Capabilities**: every kernel object requires a valid capability with explicit rights
- **W^X**: writable pages automatically get NX; `SYS_PROTECT` enforces transitions
- **ASLR**: stack randomized with RDTSC-seeded xorshift64 (0–64 KB jitter)
- **Stack canaries**: `-Z stack-protector=strong` on all binaries
- **Multiuser**: SHA-256 password hashing, UID/GID permission model, chmod/chown syscalls
- **WASM SFI**: metered execution with instruction limits and bounded memory access

16 Implementation Statistics

Component	**Lines of Rust**
Kernel (sched, IPC, caps, mm, syscall, watchdog, migrate, user)	∼6,000
sotos-common (ABI + syscall wrappers)	∼950
sotos-wasm (WASM interpreter + SFI)	∼1,800
sotos-net (IPv4/IPv6, TCP/UDP, TLS crypto, HTTP)	∼3,500
sotos-objstore (filesystem + distributed VFS)	∼2,000
sotos-nvme (NVMe SSD driver)	∼900
sotos-xhci (USB xHCI + hub + hotplug)	∼2,000
sotos-ahci (AHCI/SATA driver)	∼1,200
sotos-audio (AC97 audio driver)	∼800
sotos-usb-storage (USB BBB + SCSI)	∼700
sotos-mouse (PS/2 + USB HID mouse)	∼800
sotos-gui (window compositor)	∼600
sotos-lua (Lua interpreter)	∼1,200
sotos-pkg (package manager)	∼400
Services (init, shell, kbd, net, nvme, xhci, vmm, hello)	∼8,500
Total	∼31,350

Table 4: Approximate lines of code by component.

17 Conclusion

sotX demonstrates that a formally-verifiable microkernel can scale to a full-featured operating system while maintaining capability-based security and per-core architecture. Key contributions:

- **Per-core IPC** reduces lock contention by $C\times$ on C-core systems
- **WASM SFI runtime** with metered execution provides process isolation without hardware page tables
- **Full driver stack**: NVMe (interrupt-driven), AHCI/SATA, USB mass storage, AC97 audio, xHCI (multi-device, hub, hotplug), PS/2+USB mouse
- **Complete networking**: IPv4/IPv6, TCP with Reno congestion control and window scaling, SHA-256/ChaCha20/X25519 crypto, HTTP/1.1 client
- **Distributed computing**: network-transparent IPC routing, live process migration with checkpoint/restore, distributed VFS with conflict resolution

Feature	Count
Syscall numbers defined	44
Capability object types	10
Userspace processes	8
Device driver libraries	9
Shell builtins	40+
WASM opcodes supported	50+
Network protocols	10 (Eth, ARP, IPv4, IPv6, ICMP, UDP, TCP, DNS, DHCP, HTTP)
Crypto algorithms	4 (SHA-256, HMAC, ChaCha20, X25519)
TLA+ specifications	3
Stack canary check sites	∼1,800/binary

Table 5: Feature counts.

- **Linux-ABI shell**: job control, command history, tab completion, redirects, here-docs, shell functions, 40+ builtins
- **musl-static binary execution**: real Linux binaries (compiled with musl-libc) run under LUCAS via syscall translation, atomic thread redirect, and SSE enablement
- **Embedded applications**: Lua 5.4 interpreter, GUI window compositor, package manager
- **Security**: W^X enforcement, ASLR, stack canaries, multiuser accounts with SHA-256 auth, chmod/chown
- **Formal verification**: TLA+ specifications of IPC, capabilities, and scheduler with safety proofs

The system comprises over 31,000 lines of Rust across 15+ crates, all compiled with `-Z stack-protector=strong` and targeting `x86_64-unknown-none`.

Future work includes:

- Self-hosting (port a language toolchain to sotX)
- Custom bootloader replacing Limine (pure 64-bit long mode from scratch)
- GPU/NPU driver integration for real heterogeneous dispatch
- Mechanized proofs using Coq or Lean for the verified kernel subset